

PATTERNS OF ARRAY

Teacher's
Sign /
Remarks

→ Are not new but
different method
to solve a
problem

1. Two pointers

2. Sliding window

3. Prefix sum approach

4. Kadane's Algorithm

Two pointers (Array/string)

- Two pointers is an algorithmic technique where we use two variables/pointers to traverse an array, string or linked list.
- Instead of repeatedly checking many elements, using nested loops, we move the pointers intelligently.
- It can reduce the time complexity of many problems from $O(N^2)$ to $O(N)$.

Basic Idea:

$L \rightarrow$ $\leftarrow R$
[1, 2, 3, 4, 5, 6]

L = left pointer

R = right pointer

Depending on the problem, one
or both pointers are moved.

why do we use Two pointers?

G. Brute force:

Suppose we want to find two numbers whose sum is target.

CODE:-

```
for i=0; i<n-1; i++
    for j=i+1; j<n; j++
        check arr[i]+arr[j]
```

Time:-

$O(n^2)$

(Two pointers:

If an array is sorted:

[1, 2, 3, 4, 5, 6, 9]

↑
L

↑
R

We can decide which pointer to move.

Time:-

$O(n)$

main benefit:-

We eliminate unnecessary comparison by using information about the array.

Most important requirement

↳ Two pointers are not automatically applicable to every array problem.

- The technique works when pointer movement can be justified.
- The most common situation is:

Sorted array:

Example:

[1, 2, 3, 5, 7, 9]

Because the values are ordered, we know

• moving left $\rightarrow$ right $\rightarrow$ value increase

• moving right $\rightarrow$ left $\rightarrow$ value decrease

This allows us to make decisions.

Two main types of Two pointers

Type - ①: Converging pointers
(Opposite direction)

↳ pointers start at opposite ends

[1 2 3 4 5 9]

↑

h

↑

r

usually:

int left = 0;

int right = arr.length - 1;

used for:

- Two sum in sorted array
- Palindrome
- Compare with another
- 3Sum
- Pair Problem
- Reversing array
- Comparing elements from both ends

CORE logic $\rightarrow$ converging / opposite direction pointers

$\hookrightarrow$ for a sorted array & a target sum

there are 3 cases

case ①: $sum == target$

$arr[left] + arr[right] == target$

$\hookrightarrow$ Then Answer found.

case ②: $sum < target$

$arr[left] + arr[right] < target$

$\hookrightarrow$ we need a larger sum

since array is sorted

$left++$

case ③: $sum > target$

$arr[left] + arr[right] > target$

since array is sorted

$right--$

$\hookrightarrow$ we need a smaller sum

V/pump

Template to memorize

$int left = 0;$

$int right = arr.length - 1;$

while ($left < right$)

{

$int sum = arr[left] + arr[right]$

if ($sum == target$)

{

}

else if ($sum < target$)

{

$left++$;

}

else

{

$right--$;

}

}

$\rightarrow$ This condition will get false when both the pointers cross each other

Notes

Too small $\rightarrow left++$

Too big $\rightarrow right--$

correct $\rightarrow$ answer

Type - ②

$\hookrightarrow$ Both

low

high

Example

[1, 2, 3, 4, 5]

Two

Notes: 98

• 40

How to read problem

clue - ①: The

clue - ②: The

clue ③:

clue ④:

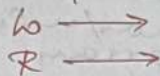
clue ⑤:

clue ⑥:

clue ⑦:

Type - (2) : PARALLEL POINTER (Same direction)

Both pointers move in the same direction



Example:

[1, 1, 2, 2, 3]



This is commonly used for:

- Removing Duplicate
- Moving Zeroes
- Palindrome
- Sorting elements
- In-place array modification

Note: If the array is unsorted

- You cannot blindly use the opposite pointer technique

You have two common options

Option (1) - Hashmap $O(N)$

Option (2) - Sort + Two pointers $O(N \log N)$

How to recognise two pointer problem

clue (1) : The problem contains

clue (2) : The problem asks for

clue (3) : " (find two no. satisfying a condition)

clue (4) : " (both ends of an array/string)

clue (5) : " (reverse/compare/move elements)

clue (6) : " (removing duplicate in place)

clue (7) : " (a sorted array + target)

When NOT to use - Two point.

- The array is not sorted & there is no regular way to sort it
- Pointer movement can't be logically anticipated.
- The problem asks for max/min sub array

→ Random

Template to memorize:

```

int left = 0;
for (int right = 0; right < arr.length; right++)
{
    // process arr[right]
    // process / move left
    left++;
}

```

3. Important idea:

right → explore

left → maintain the valid / desired pointer.

Type - ③ - fast slow pointer

→ there is a special form of same direction pointer

- Both start at the beginning, but they move at different speeds

slow →

fast → →

Template:

```

int slow = 0;
int fast = 0;
while (fast < arr.length)
{
    // process
    // move slow
    // move fast
}

```

3.

→ the

Type - ④ -

→ there is a special form of same direction pointer

→

→

Template for

```

else
{
    right = ...
}

```

Template:

```

int slow = 0;
int fast = 0;
while (fast < arr.length)
{
    // use slow & fast
    slow++;
    fast += 2;
}

```

→ This is game with linked list.

Type - (H) - fixed + two pointers

→ This is important for sum, ksum etc.

Suppose:

arr = [-1, 0, 1, 2, -1, -4]

→ we get an element

[-1, 0, 1, 2, -1, -4]

↑

→ then we take pointers on the remaining part:

[-1, 0, 1, 2, -1, -4]

↑
left

↑
right

Template:

```

for (int i = 0; i < n - 2; i++)
{

```

```

    int left = i + 1;

```

```

    int right = n - 1;

```

```

    while (left < right)
    {

```

```

        // use arr[i], arr[left], arr[right]
        if (sum == target)
        {

```

```

            // found
        }
        else
        {

```

```

            if (sum < target)
            {

```

```

                right--;
            }
            else
            {

```

```

                left++;
            }
        }
    }
}

```